Reproducibility & project workflow in Python

ZAS Pythonistas working group

May 2026

Session overview

Duration: 90 minutes. **Format:** Discussion + live demo + hands-on exercise.

This session builds from general principles to Python-specific tools – concepts that apply in any language first, then the specific tools and file formats you will use day-to-day in Python.

Schedule

Table 1

Section	Scope	Time
1. What is reproducibility?	Language-agnostic	8 min
2. Virtual environments & file paths	Python-specific	18 min
3. Naming conventions & folder structure	Language-agnostic	12 min
4. Version control with git	Language-agnostic	10 min
5. Quarto vs. Jupyter notebooks	Python-specific	17 min
6. Hands-on exercise	Python	18 min
7. Wrap-up & next topic vote	–	7 min

---

1 - What is reproducibility?

Reproducibility means someone else – or future you – can re-run your analysis and get the same result. It operates at two levels:

- **Environment reproducibility:** the same packages and versions are installed. Your code does not break because a collaborator has a different version of pandas.
- **Workflow reproducibility:** the same code runs in the same order and produces the same outputs. No mystery variables, no deleted raw files, no steps that exist only in someone's memory.

Neither layer is sufficient alone. A perfectly locked environment does not help if the script requires you to run cell 7 before cell 3. Clean, ordered code does not help if it requires a package version nobody can install anymore.

Common failure modes: "it works on my machine" *(environment)*; raw data was overwritten and the analysis cannot be re-run *(workflow)*; a notebook gives different results depending on the order cells were run *(workflow)*; a script uses an absolute path like `/Users/name/projects/...` that breaks on every other machine *(workflow)*.

Reproducibility is a spectrum – the goal is not perfection from day one, but deliberate choices that reduce friction for your future self and collaborators.

2 - Virtual environments & file paths

Virtual environments

A virtual environment is an isolated Python installation scoped to a single project – the R equivalent of an `renv` library. Do not install packages into your global Python.

This working group uses conda. venv + pip is shown as a lighter alternative for pure-Python projects.

Table 2

	conda (current setup)	venv + pip (alternative)
Create an environment	conda create -n my-env python=3.11	python -m venv .venv
Install packages	conda install pandas or pip install pandas	pip install pandas
Record env for sharing	conda env export –from-history > environment.yml	pip freeze > requirements.txt
Restore environment	conda env create -f environment.yml	pip install -r requirements.txt
Lockfile (approx. renv.lock)	environment.yml or conda-lock for full pinning	requirements.txt or pyproject.toml + uv.lock
When to use	Non-Python deps, HPC clusters, mixed R+Python	Pure-Python projects, lighter setup

`--from-history` vs full export: `conda env export` pins every package including transitive dependencies and platform-specific build strings, making it non-portable across operating systems. `--from-history` records only what you explicitly installed, letting conda resolve the rest per platform. For sharing within a team, `--from-history` is usually the better choice.

A note on terminals: Mac vs Windows

All code in this handout is written for **bash** (the shell used on macOS and Linux). Windows uses PowerShell by default, which has different syntax. Windows users should use one of the following so the commands here work as written:

Table 3

Option	Description	For this session
Git Bash	Installs with Git for Windows. Gives a full bash shell. Simplest option for this session.	Yes – simplest
WSL (Windows Subsystem for Linux)	A full Linux environment inside Windows. More powerful but requires one-time setup.	Optional
Positron / VS Code integrated terminal	Auto-uses bash or Git Bash if configured. Conda commands work out of the box.	Yes
Anaconda Prompt	A Windows terminal pre-configured for conda. Conda commands work but bash syntax may not.	Conda only

Basic conda workflow:

Listing 1 `Terminal (bash / Git Bash / WSL)`

```
conda create -n zas-pythonistas python=3.11   # create once
conda activate zas-pythonistas                 # activate
conda install pandas matplotlib                # install
conda env export --from-history > environment.yml  # record
conda env create -f environment.yml            # restore (others)
```

Positron / VS Code: Command Palette -> *Python: Select Interpreter* -> choose your conda environment. The integrated terminal will use your active conda environment automatically.

IDEs: VS Code and Positron

Both VS Code and Positron are built on the same engine (VS Code's core), so the interface is nearly identical. Positron is Posit's fork tuned for data science: it adds a persistent Variables pane, a Console panel, and a Plots viewer – all familiar from RStudio. For this working group either IDE works; Positron is recommended if you are coming from RStudio.

Table 4

	VS Code	Positron
Built on	VS Code core	VS Code core (Posit fork)
Python interpreter selection	Command Palette -> Python: Select Interpreter	Same – Command Palette -> Python: Select Interpreter
Environment activation	Auto-activates .venv or conda env in workspace	Same – auto-activates on project open
Variables pane	Via Python extension (limited)	Built-in, always visible (like RStudio)
Console panel	Integrated terminal only	Built-in, persistent (like RStudio)
Plots viewer	Via Jupyter extension	Built-in viewer pane
Notebook support	Full .ipynb support	Full .ipynb support
R support	Via R extension	First-class, built-in
Recommended for	General development, large teams	Data science, R users moving to Python

uv – a faster modern alternative worth knowing about

`uv` is a package and project manager for Python written in Rust (by the team behind `ruff`). It replaces venv, pip, pip-tools, and pyenv with a single tool and is 10-100x faster than pip for installs.

What makes it different from venv + pip:

- Generates a proper lockfile (`uv.lock`) with full dependency graph resolution – a much closer analogue to `renv.lock` than `pip freeze` is
- Manages Python versions itself: `uv python install 3.12` – no separate pyenv needed
- Scaffolds new projects: `uv init my-project`
- Runs scripts without manual activation: `uv run script.py`

Basic workflow (Terminal):

```
uv init my-project        # scaffold project + pyproject.toml
cd my-project
uv add pandas matplotlib  # install + update lockfile automatically
uv sync                   # restore environment from uv.lock
```

See: https://docs.astral.sh/uv

Project-relative file paths

Hardcoded absolute paths break on every machine except the one they were written on. The fix is to construct all paths relative to the project root.

Listing 2 `Python`

```python
from pathlib import Path

# __file__ is the path of the current script
# .parent navigates up one folder at a time
PROJECT_ROOT = Path(__file__).parent.parent

data_path   = PROJECT_ROOT / "data" / "raw" / "stimuli.csv"
output_path = PROJECT_ROOT / "outputs" / "figures"
output_path.mkdir(parents=True, exist_ok=True)  # create if needed
```

This is the Python equivalent of `here::here()` in R. The path resolves correctly regardless of where the script is called from.

---

3 - Naming conventions & folder structure

These conventions apply in any language. Consistent structure makes projects navigable by collaborators and comprehensible to your future self.

Recommended folder layout

Listing 3 `Project structure`

```
my-project/
  data/
    raw/          # original data -- never modified
    processed/    # derived data, reproducible from raw
  scripts/        # numbered analysis scripts
  notebooks/      # exploratory .ipynb files
  outputs/
    figures/
    tables/
  docs/           # manuscripts, protocols, notes
  environment.yml # conda environment (or requirements.txt)
  .gitignore
  README.md
```

Raw data is sacred. Never overwrite it, never modify it in place. All transformations happen in code and produce outputs in `data/processed/`.

File naming

Table 5

Rule	Avoid	Prefer
Lowercase only	My Analysis.py	my-analysis.py
No spaces	model results final.csv	model-results-final.csv
Date-prefix versioned outputs	results.csv	2025-05-04_model-results.csv
Verb-first for scripts	data.py	01_clean-data.py
Zero-pad numbered scripts	1_clean.py, 10_model.py	01_clean.py, 10_model.py
Be specific	analysis.py, data2.csv	01_clean-eyetracking.py

README as a map

Every project should answer three questions: what does this project do, how do I run it, and what do the outputs mean?

Listing 4 `README.md`

```
# Project name
One sentence describing the study or task.

## Setup
conda env create -f environment.yml
conda activate zas-pythonistas

## Running the analysis
python scripts/01_clean-data.py
python scripts/02_fit-model.py

## Outputs
outputs/figures/  -- all plots, referenced in the manuscript
outputs/tables/   -- model results tables
```

4 - Version control with git

Git tracks changes to your files over time, so you can see what changed, when, and why – and roll back if something breaks. It is language-agnostic and works for any text-based project. GitHub (and GitLab, Codeberg, etc.) are hosting platforms for git repositories; git itself runs entirely on your local machine.

Core concepts

- **Repository (repo):** a folder tracked by git, containing your project and its full history
- **Commit:** a saved snapshot of your project at a point in time, with a message describing what changed
- **Staging area:** a holding zone where you choose exactly which changes to include in the next commit – you do not have to commit everything at once
- **Branch:** a parallel version of the project where you can develop a feature or fix without affecting the main version (more on this below)

The basic workflow

Listing 5 `Terminal`

```
# --- One-time setup ---
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"

# --- Start tracking a project ---
git init                        # initialise a new repo in the current folder

# --- Day-to-day workflow ---
git status                      # see what has changed
git add scripts/01_analyse.py   # stage a specific file
git add .                       # stage everything (use carefully)
git commit -m "fix: use relative paths in analysis script"

# --- Review history ---
git log --oneline               # compact list of past commits
git diff                        # see unstaged changes line by line
```

Writing useful commit messages

A good commit message completes the sentence "If applied, this commit will…". Use a short imperative verb phrase: `add`, `fix`, `update`, `remove`. Keep it under 72 characters.

Table 6

Avoid	Prefer
fixed stuff	fix: use relative paths in analysis script
update	add: conda environment file and README setup instructions
changes	remove: hardcoded absolute paths from all scripts
asdfgh	refactor: split data cleaning into separate script

What to put in .gitignore

Listing 6 `.gitignore`

```
# Python
.venv/
__pycache__/
*.pyc
*.pyo
.ipynb_checkpoints/

# Conda
.conda/

# Secrets -- never commit these
.env
*.key

# Large data files (store elsewhere, e.g. OSF or NAS)
data/raw/
*.csv
*.edf
*.mat
```

A note on branching

Branches let you work on a new feature or experiment without touching the main version of your project. When the work is ready, you merge it back. For solo projects, you may not need branches at all – a clean linear commit history is fine. For collaborative projects, a simple convention is: keep `main` stable and do new work on a named branch.

Listing 7 `Terminal`

```
git branch feature/add-preprocessing   # create a branch
git checkout feature/add-preprocessing # switch to it
                                        # (or: git checkout -b name, to do both at once)
git checkout main                       # switch back to main
git merge feature/add-preprocessing    # merge when ready
```

We will cover branching and collaborative workflows (pull requests, merge conflicts) in a future session.

5 - Quarto vs. Jupyter notebooks

They are not competitors

Quarto *uses* Jupyter as its execution engine – adding `jupyter: python3` to a `.qmd` header tells Quarto to call Jupyter to run your code. The real choice is between two **file formats**:

Table 7

	.ipynb notebook	.qmd (Quarto)
File format	.ipynb (JSON)	.qmd (plain text)
Execution order	Any order – cells run independently	Always top-to-bottom, fresh process
Kernel state	Persists between cells; can be stale	No persistent state between renders
Git diffs	Noisy – outputs embedded in JSON	Clean – plain text only
Output storage	Stored inside the file	Separated from source
Multi-format export	Requires nbconvert + manual steps	Built-in: HTML, Word, PDF, slides
Inline computed values	Not straightforward	‘python round(val, 2)‘
Best for	Exploration, iteration, teaching	Reports, papers, shared outputs

The hidden state problem in notebooks

The biggest reproducibility risk in `.ipynb` files is **out-of-order execution**. Jupyter remembers every variable set during a session, even from cells you have deleted or moved. A notebook that appears to work may depend on a variable set hours ago in a cell that no longer exists. The only reliable check is *Kernel -> Restart & Run All* before sharing – and people often forget.

What `freeze: true` does in Quarto

By default, Quarto re-runs all your code from scratch on every render – correct for reproducibility, but slow for heavy computations like model fitting or EEG preprocessing. `freeze: true` changes this: Quarto **caches each chunk’s output** and only re-runs a chunk when its source code actually changes.

- **First render:** runs everything, saves outputs into a `_freeze/` folder alongside your `.qmd`
- **Subsequent renders:** skips unchanged chunks and reuses saved outputs
- **After editing a chunk:** only that chunk re-runs

Committing `_freeze/` to git means collaborators can render the document and see your outputs without needing your data or compute environment.

Listing 8 `_quarto.yml`

```yaml
execute:
  freeze: true
```

> **R parallel:** `freeze: true` is roughly analogous to `knitr cache = TRUE` in R Markdown, but more reliable – it caches at the file level, tracks source changes explicitly, and plays well with git.

A practical two-stage workflow

Use notebooks for the messy exploratory work. Once the approach is settled, move clean code into a `.qmd` for the shareable output.

Listing 9 `Terminal`

```
# Extract clean code from a notebook when you are ready
jupyter nbconvert --to script exploration.ipynb
# produces exploration.py -- paste the useful parts into your .qmd
```

6 - Hands-on exercise

Setup: the group repo is already cloned and your conda environment is active. The repo contains `scripts/01_analyse.py` (a short script with a reproducibility problem) and `environment.yml`.

Part A - Dependency management

Table 8

Step	Task
A1	Activate your conda environment: conda activate zas-pythonistas
A2	Export it: conda env export > environment-full.yml – open the file and inspect what is there
A3	Compare with the leaner version: conda env export –from-history > environment-minimal.yml – what is the difference?
A4	Optional: try the venv equivalent in a separate terminal: python -m venv .venv && source .venv/bin/activate && pip install pandas && pip freeze > requirements.txt

Part B - Project-relative file paths

Open `scripts/01_analyse.py`. It contains a hardcoded absolute path that breaks on every machine except the one it was written on.

Table 9

Step	Task
B1	Run the script and read the error: python scripts/01_analyse.py
B2	Fix the broken path using pathlib so it resolves relative to the project root (see the pathlib pattern on this handout)
B3	Run the script again to confirm it works: python scripts/01_analyse.py

Part C - Git hygiene

Table 10

Step	Task
C1	Check what git would currently commit: git status – notice anything that should not be there?
C2	Create a .gitignore that excludes: __pycache__/, *.pyc, .ipynb_checkpoints/, .venv/ (if created in A4)
C3	Stage your fixed script and .gitignore and commit with a descriptive message
Stretch	Move the output of 01_analyse.py into a .qmd file and render it to HTML with Quarto

Skeleton script for the repo

Add this as `scripts/01_analyse.py` before the session. Also commit a small `data/raw/sample.csv` with a few rows of dummy data.

Listing 10 `scripts/01_analyse.py`

```
import pandas as pd
from pathlib import Path

# ------------------------------------------------------
# TODO: this path is broken -- fix it using pathlib
# ------------------------------------------------------
data_path = "/Users/replace-me/projects/zas-pythonistas/data/raw/sample.csv"

df = pd.read_csv(data_path)
print(f"Loaded {len(df)} rows")
print(df.describe())
```

R to Python parallels

Table 11

Concept	R	Python
IDE	RStudio / Positron	Positron / VS Code
Isolated environment	renv library (.Rprofile)	conda env or .venv (venv)
Package install	install.packages() / pak::pak()	conda install / pip install
Dependency lockfile	renv.lock	environment.yml or requirements.txt or uv.lock
Restore environment	renv::restore()	conda env create -f environment.yml or pip install -r requirements.txt
Project root	.Rproj file sets working directory	No automatic root – use explicit Path(__file__) convention
Project-relative paths	here::here()	Path(__file__).parent / ...
Set random seed	set.seed(42)	random.seed(42) + numpy.random.seed(42)
Dynamic report format	.Rmd or .qmd	.qmd with jupyter: python3
Freeze / cache computations	knitr cache = TRUE	freeze: true in Quarto
Inline computed value	‘r round(val, 2)‘	‘python round(val, 2)‘
Version control	git (same)	git (same)

Resources

- **pythRon book** (ZAS internal): R/Python concepts side-by-side with worked examples
- **Quarto + Python:** quarto.org/docs/computations/python.html
- **Quarto freeze:** quarto.org/docs/projects/code-execution.html#freeze
- **uv:** docs.astral.sh/uv
- **Pro Git book (free):** git-scm.com/book/en/v2
- **Real Python – virtual environments primer:** realpython.com/python-virtual-environments-a-primer
- **The Turing Way** – community handbook for reproducible research: the-turing-way.netlify.app